Smart Environment Monitoring & Control System

SWAPD 453: IoT Applications Development
Semester: Spring 2026
Assignment: 1

Team Members:

- Nour Awad 202200626
- Merna Ahmed 202201530

Smart Environment Monitoring & Control System

1. System Architecture Diagram

The system follows a layered IoT architecture composed of sensing, communication, and application layers.

1. **Sensing Layer** – Sensors connected to the ESP32 collect environmental data including temperature, humidity, motion, light level, and distance.
2. **Communication Layer** – The ESP32 sends sensor data to an MQTT broker over WiFi and receives actuator commands from Node-RED.
3. **Application Layer** – Node-RED processes the data, displays it on a dashboard, and allows users to control actuators remotely.

Data flow:

Sensors → ESP32 → MQTT Broker → Node-RED Dashboard
Dashboard → MQTT Commands → ESP32 → Actuators

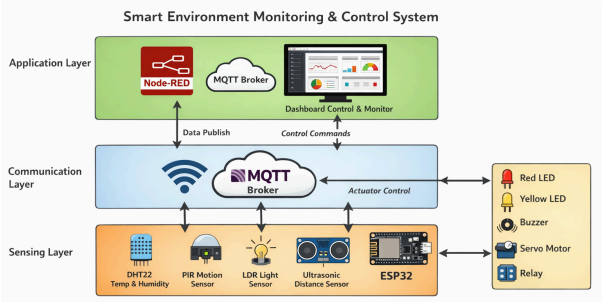


Fig.1: System Architecture Diagram

---

2. System Description

The Smart Environment Monitoring and Control System is an IoT-based application designed to monitor environmental conditions and automatically control devices.

The system uses an **ESP32 microcontroller** connected to several sensors:

- **DHT22** for temperature and humidity monitoring
- **LDR sensor** for light level detection
- **PIR sensor** for motion detection
- **Ultrasonic sensor** for distance measurement

The collected data is transmitted through **WiFi using the MQTT protocol** to a **Node-RED dashboard**, where users can monitor sensor readings in real time.

The system also includes several actuators:

- **Red LED** indicates high temperature
- **Yellow LED** indicates dark conditions
- **Buzzer** activates when motion is detected
- **Servo motor** moves when an object is detected nearby
- **Relay module** activates when temperature exceeds a threshold

The system performs **automatic control based on predefined thresholds**, while also allowing **manual control through the Node-RED dashboard**.

---

3. Challenges Faced

Several challenges were encountered during the implementation of the system.

- **Communication errors with the DHT22 sensor**, which caused repeated timeout errors. This was resolved by correcting the wiring configuration and ensuring the correct GPIO pin was used.
- **MQTT connection failures**, which occurred due to incorrect broker port configurations. Switching to the correct MQTT port (1883) resolved the issue.
- **GPIO pin management** to avoid conflicts and ensure reliable readings.

---

4. Lessons Learned

- IoT system architecture
- MQTT communication protocols

- ESP32 sensor integration
- Real-time dashboard development
- Designing automatic control systems
- Implementing remote device control via Node-RED

5. Screenshots

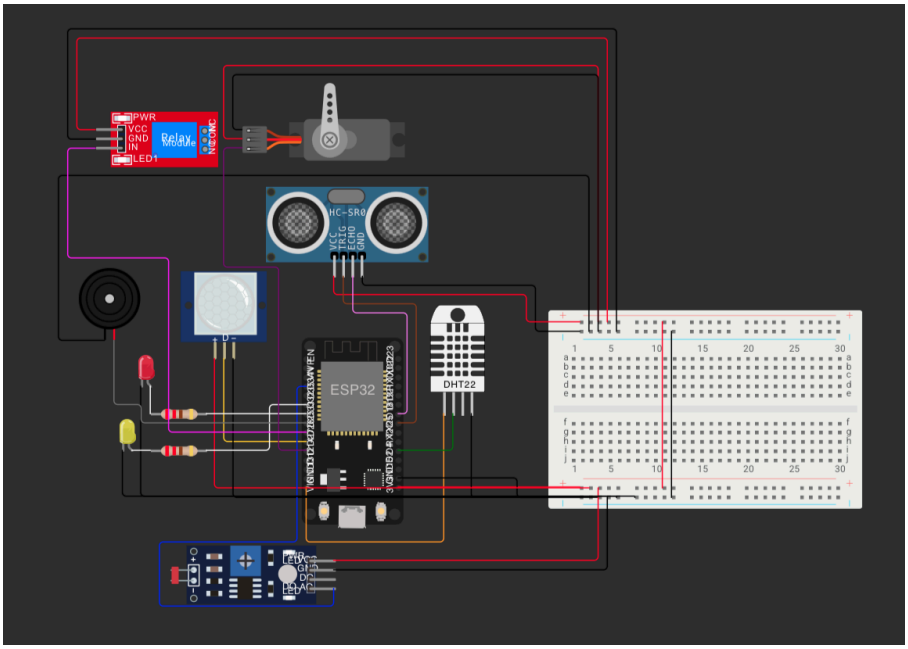
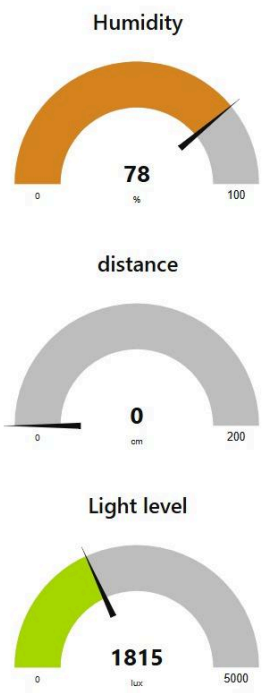
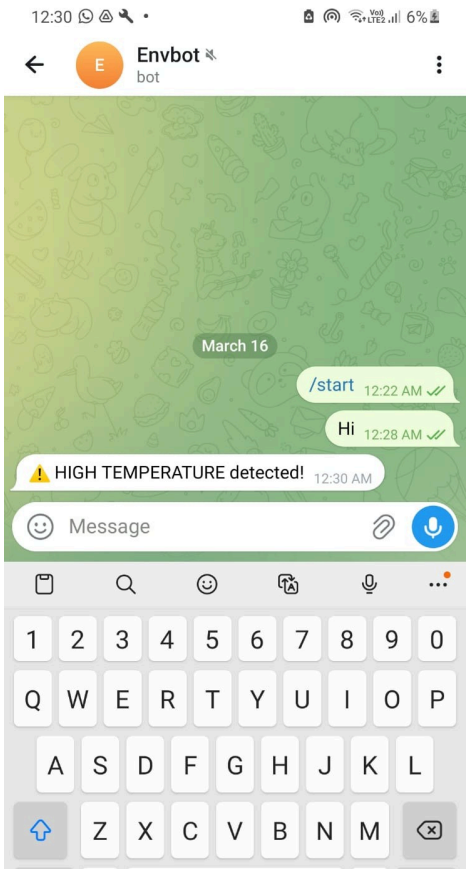
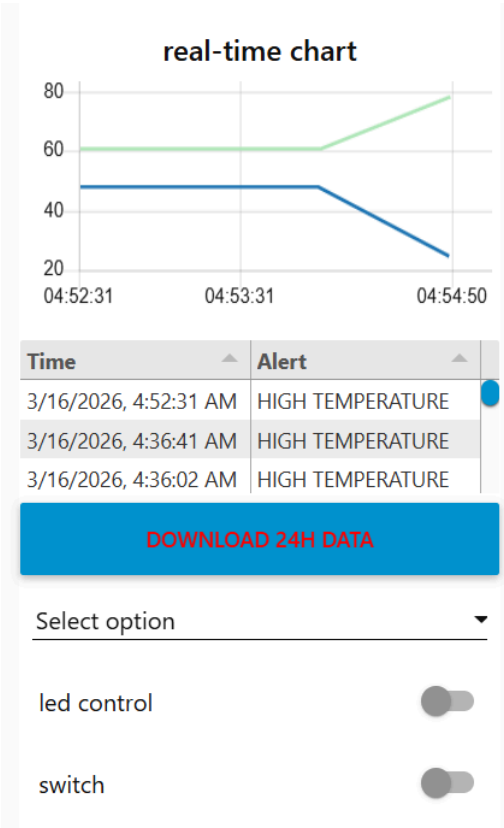


Fig 2: Wiring Diagram

Dashboard screenshots